



# مشكلة البائع المتجول

دراسة المشكلة ، خوارزميات الحل،  
والمقارنة بين الطرق المختلفة

## يتضمن:

- تعريف المشكلة والساريو
- التعقيد الحسابي للمشكلة
- مدخل إلى خوارزمية LKH-3
- The Alpha ( $\alpha$ ) Transformation
- Lagrangian Relaxation
- Subgradient  $G_i$
- The Cycle-Exchange Theorem
- ابتكار Lin-Kernighan: Lin-Kernighan
- التبديل التسلسلي



## إعداد:

- أسامة محمد
- أحمد الخليل
- أنس العبد الكريم
- ثابت الهويدي
- اليمان حميد



فهم شامل  
للمشكلة



طرق حل  
فعالة



تحليل  
دقيق



مقارنة بين  
الخوارزميات



نتائج وتوصيات  
أفضل



# Travelling Salesman Problem (TSP)

By

Osama Mohamed, Ahmed Al-khlil, Anas Al-abdolkarem,  
Thabet Hameed, and Alyman Hamied

Department of Computer Engineering and Automation, Damascus University  
H.M.K Team

June 22, 2026

لا تزال مسألة البائع المتجول (TSP) تمثل تحديًا أساسيًا في مجال التحسين التوافقي والتوجيه اللوجستي نظرًا لتعقيدها الحسابي من فئة NP-Hard. ورغم أن الخوارزميات الدقيقة تضمن الوصول إلى الحل الأمثل المطلق، إلا أنها لا تتناسب مع التطبيقات واسعة النطاق في العالم الحقيقي، وتصبح غير قابلة للتطبيق حسابيًا. تبحث هذه الورقة في تنفيذ خوارزمية Lin-Kernighan-Helsgaun (LKH) الاستدلالية، وكفاءتها المعمارية، ومقاييس أدائها، والتي تُعتبر على نطاق واسع واحدة من أقوى أطر البحث الموضوعي لتقريب حلول TSP المثلى. من خلال الاستفادة من آليات تبديل الوصلات التكميلية المتقدمة من نوع k-opt، يوازن إطار عمل LKH بين القيود الهيكلية والتقارب الخوارزمي السريع. نقوم بتقييم حدود تنفيذها ودقتها عبر سيناريوهات توجيه معيارية محددة، مما يُظهر قدرتها على تقريب أطوال المسارات شبه المثلى بشكل موثوق مع الحد الأدنى من الحمل الحسابي. تؤكد النتائج على متانة الخوارزمية وقابليتها للتوسع في بيئات الأتمتة المعقدة.

# المحتويات

|           |                                                                             |
|-----------|-----------------------------------------------------------------------------|
| <b>5</b>  | <b>1 تعريف المشكلة</b>                                                      |
| 5         | 1.1 السيناريو                                                               |
| 5         | 2.1 التعقيد الحسابي للمشكلة                                                 |
| 6         | 3.1 انواع المشكلة وعدد الوصلات                                              |
| 6         | 4.1 طرق حساب وتمثيل البيانات                                                |
| 7         | 5.1 اختيار الخوارزميه المناسبه                                              |
| 7         | 1.5.1 معايير المقارنه في اختيار الخوارزميه المناسبه                         |
| 7         | 2.5.1 مقارنه الخوارزميات                                                    |
| <b>10</b> | <b>2 خوارزميه LKH-3</b>                                                     |
| 10        | 1.2 مدخل الى عبقرية خوارزميه LKH-3                                          |
| 10        | 2.2 الخطوة الاولى: تهيئه البيانات                                           |
| 10        | 1.2.2 حساب المسافات ونخزينها                                                |
| 11        | 2.2.2 طريقه تمثيل بيانات العقد في الرسم البياني                             |
| 11        | 3.2 الخطوة الثانيه: The Held-Karp optimaization loop                        |
| 11        | 1.3.2 القاعده الرياضيه: Lagrangian Relaxation                               |
| 13        | 2.3.2 حساب شجره 1-Tree                                                      |
| 15        | 3.3.2 حساب $G_i$ Subgradient                                                |
| 17        | 4.3.2 حساب حجم الخطوة $\lambda$ Step Size                                   |
| 18        | 5.3.2 تحديث قيم العقاب $\pi$                                                |
| 18        | 4.2 الخطوة الثالثه: The Alpha ( $\alpha$ ) Transformation and Candidate Set |
| 18        | Generation                                                                  |
| 19        | 1.4.2 ما هي قيم ألفا ( $\alpha$ ) ؟                                         |
| 19        | 2.4.2 التعريف الرياضي لقيمه التقارب (Nearness Value)                        |
| 19        | 3.4.2 التفسير الطوبولوجي لقيم ألفا: Topological Interpretation              |
| 20        | 4.4.2 الآليه الخوارزميه لحساب قيم ألفا: The Cycle-Exchange Theorem          |
| 21        | 5.4.2 استراتيجيه الحساب المسبق لأكبر الوصلات في اي مسار فريد                |
| 22        | 6.4.2 تبسيط الرسم البياني وبناء مجموعات المرشحين                            |
| 22        | 7.4.2 التعريف الرياضي لمجموعه المرشحين                                      |
| 23        | 8.4.2 اختيار حد عدد المرشحين: The Theoretical Bound                         |
| 23        | 5.2 الخطوة الرابعه: محرك البحث المحلي المتغير k-opt                         |
| 23        | 1.5.2 الهدف من البحث الملحي (Local Search)                                  |
| 23        | 2.5.2 كيف يعمل البحث الملحي مع تبديلات opt-2, top-3, ..., opt-k             |
| 24        | الثابته                                                                     |
| 25        | 3.5.2 قيود خوارزميات k-opt الثابته                                          |
| 26        | 4.5.2 ابتكار Lin-Kernighan: التبديل التسلسلي ذو العمق المتغير               |
| 29        | 6.2 Backtracking and the LKH-3 Extension                                    |

|           |                                                                                 |          |
|-----------|---------------------------------------------------------------------------------|----------|
| <b>30</b> | <b>Real-World Applications of LKH-3</b>                                         | <b>3</b> |
|           | 1.3 الخدمات اللوجستية في تجاره الالكترونيه وتحسين النقل (Amazon, FedEx, UPS)    |          |
| 30        | .....                                                                           |          |
| 31        | 2.3 تصنيع أشباه الموصلات وتحديد مسارات حركة الروبوتات (Intel, TSMC)             |          |
|           | 3.3 خدمات حجز المركبات ووسائل التنقل حسب الطلب ديناميكياً (Uber, Grab, Meituan) |          |
| 31        | .....                                                                           |          |
|           | 4.3 المعلوماتية الحيوية المتقدمة و Genome Assembly (المرافق المركزية لعلم       |          |
| 32        | ..... (Genome                                                                   |          |
| <b>32</b> | <b>Conclusion: The Engineering Legacy of Lin-Kernighan</b>                      | <b>4</b> |

# 1 تعريف المشكلة

## 1.1 السيناريو

تخيل أن هناك شركة توصيل لوجستية تمتلك مركبة توصيل واحدة، وانطلاقاً من المقر الرئيسي للشركة، يتعين على السائق زيارة عدد محدد من المدن (أو العملاء) لتسليم البضائع، ثم العوده في نهايه المطاف الى نقطه البدايه (المقر الرئيسي). ولتحقيق اعلى مستوى من الكفاءة وتقليل التكاليف الاضافيه، يواجه السائق عده شروط صارمه:

1. يجب زياره كل مدينه مره واحده فقط.
2. يجب تقليل اجمالي المسافه المقطوعه (او اجمالي زمن الرحله او الوقود) الى الحد الادنى.
3. يجب العوده الى نقطه الانطلاق (المقر الرئيسي).
4. يمنع بتاتا اغلاق حلقه جزئيه ببعض المدن ثم العوده الى نقطه البدايه والخروج مره اخرى لاكمال باقي المدن.

## 2.1 التعقيد الحسابي للمشكلة

تكمن الصعوبه البالغه في هذه المشكله عندما يبدأ عدد المدن بالارتفاع، حيث يزداد عدد المسارات الممكنه بشكل "عاطلي" ( $n!$ ). لنستعرض كيف تتضخم الاحتمالات:

- اذا كان على السائق زياره 3 مدن فقط، فان حساب المسار الاقصر سهل للغاية لان الاحتمالات محدوده.
- اذا ارتفع العدد الى 10 مدن، يقفز عدد المسارات المحتمل الى اكثر من 180000 مسار بديل.
- اما اذا كان هناك 20 مدينه فقط، فان عدد الاحتمالات ينفجر ليصل الى رقم خيالي من الكوينتيليونات ( $10^{18}$  مسار).

بسبب هذا النمو الأسّي الهائل في الاحتمالات، يصبح من المستحيل على الحواسيب العاديه حساب واختبار كل مسار على حده للوصول الى الحل المثالي مع زياده حجم البيانات. لذلك، تصنف "مشكلة البائع المتجول" (TSP) كواحد من أشهر المسائل غير الحتميه متعدد الحدود الصعبه جداً (NP-hard)، وهي المسائل التي يسهل يصاغتھا وفھمھا، ولكن يصعب جداً حلھا بشكل مطلق عندما تصبح واسعه النطاق.

### 3.1 أنواع المشكله وعدد الوصلات

تنقسم الى نوعين رئيسيين بناءً على اتجاه الطرق:

1. **مشكله البائع المتجول المتناظره Symmetric TSP**: هنا تكون الطرق ذات اتجاهين ومتساويه في التكلفة. اي ان المسافه من المدينه A الى المدينه B, تساوي المسافه من B الى A. اي  $(c_{ij} = c_{ji})$ . ويحسب عدد الوصلات بالعلاقه:

$$NumberofEdges = \frac{n(n-1)}{2} ; \quad n \in N^*$$

2. **مشكله البائع المتجول غير المتناظره Asymmetric TSP**: وهنا تكون الطرق موجهة وتختلف تكلفها حسب الاتجاه  $(c_{ij} \neq c_{ji})$ . ويحسب عدد الوصلات بالعلاقه:

$$NumberofEdges = n(n-1) ; \quad n \in N^*$$

### 4.1 طرق حساب وتمثيل البيانات

يجب حساب المسافات بين المدن وتجهيزها قبل ادخالها الى الخوارزميه كي لا تستهلك قدره المعالج في تكرار حسابات غير مهمه, ولحساب المسافات بين اي نقطتين هناك عده قوانين رياضيه, منها قانون حساب المسافه الاقليديه وهي المسافه المباشره بين اي نقطتين, وتعطى بالعلاقه التاليه:

$$d_{ij} = \sqrt{(x_i - x_j)^2 + (y_i - y_j)^2}$$

ولتمثيل البيانات رياضياً او برمجياً يتم وضع البيانات في مصفوفه ثنائيه الابعاد 2D حجمها  $n \times n$ .

• السطر يعبر عن مدينه الانطلاق, والعمود يعبر عن مدينه الوصول.

- التقاطع بين السطر  $i$  والعمود يعبر عن قيمه التكلفة  $c_{ij}$  (المسافه او الزمن).
- القيم على القطر الرئيسي تكون دائماً صفراً او مالا نهائيه, لمنع السائق من الانتقال من المدينه الى المدينه نفسها.

## 5.1 اختيار الخوارزميه المناسبه

### 1.5.1 معايير المقارنه في اختيار الخوارزميه المناسبه

عند التعامل مع مشكله البائع المتجول TSP. لا يوجد مفهوم "الخوارزميه الافضل مطلقاً" دون النظر الى سياق البيانات وحجم المشكله. ان عمليه اختيار الخوارزميه المناسبه تحكمها دائماً مقايضه اساسيه (Tread-off) بين ثلاثة عوامل رئيسيه وهي: الزمن المستغرق (Time), الموارد المتاحة (اي القدره الحسابيه والتخزينيه) (Calculation & Memory), والدقه المطلوبه (Accuracy).

بناءً على هذه الفلسفه, ولتحديد الخوارزميه التي تحقق التوازن الامثل بين كفاءه الاداء ودقه النتائج, قمنا باجراء مقارنه ممنهجه بين سته من ابرز الخوارزميات عالمياً, تتراوح بين الحلول الجشعه البسيطه, والبرمجيه الديناميكيه المعقده, وصولاً الى الخوارزميات الهجينه المتطوره. وفقاً لخمسه معايير محوريه:

1. فئة الخوارزميه (Algorithm class): التصنيف البنيوي للخوارزميه.
2. فلسفه اتخاذ القرار (Descision logic): الاليه التي تتبعها الخوارزميه لاستكشاف المسارات.
3. التعقيد الحسابي والزماني (Time & Space complexity): مدى استهلاك الخوارزميه للوقت وموارد النظام مع نمو عدد المدن ( $n$ ).
4. دقه الحل (Accuracy): مدى قرب مسار الناتج من الحل الاقصر المثالي.

### 2.5.1 مقارنه الخوارزميات

#### • الجار لاقرب (Nearest Neighbor):

1. فئة الخوارزميه (Algorithm class): جشعه (Greedy Heuristic).
2. فلسفه اتخاذ القرار (Descision logic): اختيار المدينه الاقرب محلياً في كل خطوه دون النظر للمستقبل.
3. دقه الحل (Accuracy): سيئه الى متوسطه (غالباً اطول بحوالي 25% من الحل المثالي).

4. التعقيد الحسابي والزمني (Time & Space complexity): التعقيد الزمني  $O(n)$ , والذاكرة  $O(n^2)$  أو  $O(n)$ .

• **هليد-كارب (Held-Karp):**

1. فئة الخوارزميه (Algorithm class): دقة فائقة (Exact/Dynamic Programming).
2. فلسفه اتخاذ القرار (Descision logic): تقسيم المشكله الى مشاكل فرعيه ابسط وتخزين حلولها لمنع تكرار العمليات الحسابيه وحساب الحل المثالي المطلق.
3. دقة الحل (Accuracy): الدقة مثاليه 100%, ولكن لاتستطيع معالجه اعداد مدن كبيره.
4. التعقيد الحسابي والزمني (Time & Space complexity): التعقيد الزمني انفجاري  $O(n^{2^{2n}})$ , والتعقيد التخزيني  $O(2^n)$  هائل جداً.

• **مستعمرة النمل (Ant Colony Optimization):**

1. فئة الخوارزميه (Algorithm class): ذكاء السراب (Metaheuristic).
2. فلسفه اتخاذ القرار (Descision logic): محاكاة سلوك النمل عبر افراز الفيرومون, والمسارات الاقصر تكتسب جذباً أقوى تدريجياً عبر ترسب الفيرومون.
3. دقة الحل (Accuracy): عاليه جداً (تقترب من المثاليه في الاحجام المتوسطه).
4. التعقيد الحسابي والزمني (Time & Space complexity): التعقيد الزمني  $O(I \times A \times n^2)$  حيث I التكرارات وA عدد النمل. والتعقيد التخزيني  $O(n^2)$ .

• **التطور الجيني (Generational Optimization):**

1. فئة الخوارزميه (Algorithm class): جينيه تطوريه (Metaheuristic).
2. فلسفه اتخاذ القرار (Descision logic): توليد مجتمع من الحلول, ثم اجراء تزاوج (Crossover), وطفرات (Mutation) للبقاء للأصلح والاقصر.
3. دقة الحل (Accuracy): متوسطه الى عاليه (تحتاج وقت طويل للتقارب).
4. التعقيد الحسابي والزمني (Time & Space complexity): التعقيد الزمني  $O(I \times P \times n^2)$  حيث I التكرارات وP حجم المجتمع. والتعقيد التخزيني  $O(nP)$ .

• **opt-2 & opt-3:**

1. فئة الخوارزميه (Algorithm class): تحسين محلي (Local Search).
2. فلسفه اتخاذ القرار (Descision logic): قطع وتوصيل الوصلات بين المدن, حافظتين في كل مره (2-opt), او ثلاثه في كل مره (3-opt). وذلك لايجاد وصلات اقصر.
3. دقة الحل (Accuracy): متوسطه الى جيده جداً.



4. التعقيد الحسابي والزمني (Time & Space complexity): التعقيد الزمني  $O(n^2)$  for opt-2 و  $O(n^3)$  for opt-3.

### • LKH-3:

1. فئة الخوارزمية (Algorithm class): تحسين محلي متطور (Advanced Metaheuristic).
2. فلسفه اتخاذ القرار (Descision logic): تقطيع وتوصيل للوصلات بشكل ديناميكي متغير تابع ل K من الوصلات, (opt-k) مع ميزات ذكية لمنع السقوط في الحلول المحلية الفرعية وتدعم القيود المعقدة.
3. دقه الحل (Accuracy): قريبه جداً من المثاليه (99% الى 100%).
4. التعقيد الحسابي والزمني (Time & Space complexity): في التعقيد الزمني, تقترب جداً من  $O(n^{2.2})$ . وفي التعقيد التخزيني  $O(n)$  ذو كفاءته عاليه جدا في اداره الذاكرة.

## مقارنة التعقيد الزمني لخوارزميات حل مشكلة البائع المتجول (TSP)

كلما كان التعقيد أقل كان الأداء أفضل لإيجاد حلول عالية الجودة خاصة مع زيادة عدد المدن (n).

| الخوارزمية                                                                                                          | نوع الخوارزمية                                           | التعقيد الزمني (زمن التنفيذ)            | تعقيد تقريبي                | ملاحظات                                                                           |
|---------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------|-----------------------------------------|-----------------------------|-----------------------------------------------------------------------------------|
| Nerest Neighbor                | (Constructive)                                           | $O(n^2)$                                | منخفض جداً                  | سريع جداً، لكنه لا يعطي حلولاً مثلى. جودة الحل تعتمد على نقطة البداية.            |
| Held-Karp                      | برمجة ديناميكية (Exact)                                  | $O(n^2 2^n)$                            | عالي جداً (أسى)             | يضمن الحل الأمثل، لكنه غير عملي للمشكلات الكبيرة ( $n > 20$ ).                    |
| Ant Colony Optimization (ACO)  | تحسينية قائمة على الطبيعة (Metaheuristic)                | $O(m n^2 t)$                            | متوسط                       | m: عدد النمل، t: عدد التكرارات. جودة جيدة، وزمن يعتمد على المعلمات.               |
| Generation Optimization (GO)   | تحسينية (Metaheuristic)                                  | $O(p n^2 t)$                            | متوسط                       | p: حجم المجتمع، t: عدد الأجيال. توازن جيد بين الجودة والوقت.                      |
| 2-opt                          | تحسين محلي (Local Search)                                | $O(n^2)$ لكل مرور (عدة مرات)            | منخفض                       | يحسن الحل الحالي بإزالة تقاطع حافتين. سريع وفعال لتحسين الحلول.                   |
| 3-opt                          | تحسين محلي (Local Search)                                | $O(n^3)$ لكل مرور (عدة مرات)            | متوسط - عالي                | أقوي من 2-opt لكنه أبطأ. يحسن الحل بجمع ثلاث حواف.                                |
| LKH-3                          | تحسينية متقدمة (Metaheuristic) (مبنية على 2-opt و 3-opt) | تقريباً $O(n^2)$ إلى $O(n^3)$ لكل تكرار | منخفض إلى متوسط (عملي جداً) | من أقوى الخوارزميات العملية لـ TSP. يعطي حلولاً قريبة جداً من المثلى بسرعة عالية. |

n: عدد المدن m: عدد النمل p: حجم المجتمع t: عدد التكرارات / الأجيال

مفتاح التعقيد: منخفض جداً، منخفض، متوسط، عالي، عالي جداً (أسى)

### ملاحظات عامة

- الخوارزميات الدقيقة (Exact) تضمن الحل الأمثل ولكن بتكلفة زمنية عالية جداً.
- الخوارزميات التحسينية (Metaheuristic) تضمن الأمثل لكنها عملية وقتيلة للتطبيق على مشكلات كبيرة.
- التعقيد الفعلي يعتمد أيضاً على التحسين الخوارزمية والمعلومات المستخدمة.

## 2 خوارزميه LKH-3

### 1.2 مدخل الى عبقرية خوارزميه LKH-3

في عام 1973, قدم العالمان Shen Lin و Brian Kernighan خوارزميه LK , والتي ابتكرت فكرة استخدام k-opt متغير. فبدلاً من تبديل عدد ثابت من الوصلات (مثل وصلتين او ثلاث في كل مره), تغير الخوارزميه عدد تبديل الوصلات بناءً على المعطيات ديناميكياً للخروج من فخ الانماط دون المستوى الامثل (Suboptimal Patterns).

في عام 2000, احدث العالم Keld Helsgaun ثوره في هذا البحث عندما قدم خوارزميه LKH. وقد اضاف محرك رياضي قوي لتوجيه محرك البحث المحلي: The Held-Karp Lower Bound. وذلك بدمج Subgradient Optimaization مع محرك تبديل الوصلات LK swapping engine, بذلك اصبحت خوارزميه LKH اقوى خوارزميه حل لمشكله TSP على الاطلاق.

في عام 2017, اصدر العالم Helsgaun خوارزميه LKH-3, حيث طور عده اصدارات من المشكله, مثل اضافته خاصيه توجيه عده بائعين, نوافذ الوقت, والتكديس, مما جعلها تتراس على عرش الخوارزميات الاكفئ في حل المشكله عند ازدياد حجمها.

اسطاعه خوارزميه LKH-3 تحقيق سرعتها بتقسيم المشكله الى اجزاء اصغر مركزه وذات كفاءه عاليه.

### 2.2 الخطوه الاولى: تهيئه البيانات

في المرحله الاولى نعد بيانات المدن ونقوم بحساب الوصلات بينها ونخزنها في Arrays تسهل التعامل معها.

#### 1.2.2 حساب المسافات ونخزينها

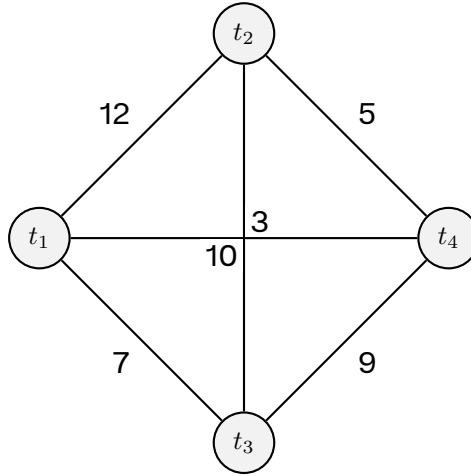
يتم حساب المسافه بين كل عقدتين (مدينتين) عبر معادله المسافه المباشره الاقليديه:

$$d_{ij} = \sqrt{(x_i - x_j)^2 + (y_i - y_j)^2}$$

وتخزن القيمه الناتجه داخل الوصله, حيث يوجد `struct Edge` يخزن معرفي العقدتين الطرفيتين الخاص بهما, والى جانبه قيمه المسافه الـ "جغرافيه", او اي واحده تكلفه مراد التعامل معها.

### 2.2.2 طريقه تمثيل بيانات العقد في الرسم البياني

اشهر الطرق في توضيح العلاقه بين العقد لحل مشكله البائع المتجول هي استخدام مايسمى بـ Weighted Graph, حيث انه بنيه بيانات مميزه, ويتكون من عقد ووصلات, كل عقده لها رقم, وكل وصله تمثل الوزن بين كل عقدتين:



### 3.2 الخطوه الثانيه: The Held-Karp optimaization loop

#### 1.3.2 القاعده الرياضيه: Lagrangian Relaxation

يمكن تمثيل مشكله البائع المتجول برقم وحيد: وهو تقليل اجمالي تكلفه الرحله مع مراعاة القيدان اللذين ينصان على ان الرحله يجب ان تتصل بجميع المدن وان تكون حلقه مفرغه واحده فقط, وان كل مدينه يجب ان يكون لها وصلتين فقط  $d_i = 2$ . ولكن بسبب اجبار هاذين القيدان في أن واحد هو ما يجعل المشكله من تصنيف (NP-Hard). تستخدم طريقه Held-Karp تكنيكاً يدعى Lagrangian Relaxation. وظيفته فصل كل قيد وتحسينه لوحده. يتم تحقيق القيد الاول باستخدام بنيه تدعى 1-Tree والتي تحقق الشروط التي تنص على ان الحلقه الصحيحه يجب ان تصل جميع المدن ببعضها ويكون بها حلقه واحده فقط بناءً على مصفوفه المسافات وبغض النظر عن درجه كل مدينه. وبدلاً من اجبار قيد الدرجه ان يكون 2 بالقوه, يقوم القيد الثاني بإلغاء الدرجات الثابته وينقلها الى داله هدف, وهي داله لا يهمها البنيه العامه الطوبولوجيه, وانما هدفها الاساسي هو تعديل قيم تعرف بـ قيم العقوبه  $\pi_i$  normalized Vales, والتي تمثل الخطأ الهيكلي برقم ضرببي والذي يمثل المسافه المعدله التي يمكن للقيد الاول فهمها.

وتهدف لايجاد (penalty). لاي مصفوفه من الارقام الحقيقيه  $\pi = (\pi_1, \pi_2, \dots, \pi_n)$  نعرف مصفوفه من قيم  $\pi$  المعدله  $C'$ :

$$C'_{ij} = C_{ij} + \pi_i + \pi_j$$

حيث  $C_{ij}$  هي المسافه الجغرافيه الحقيقيه بين المدينه  $i$  والمدينه  $j$ . والقيم  $\pi_i$  و  $\pi_j$  هي مضاعفات لاغرانج Lagrange Multipliers. اذا جمعنا هذه القيم  $C'_{ij}$  لكل مدينه, تصبح التكلفه المعدله لمجموعه مدن  $N$  تحتوي وصلات متغيره او تابعه بشكل منهجي تبعاً للدرجات الخاصه بكل مدينه  $d_{ij}$ , وتسمى Penalized Cost:

$$\begin{aligned} W(\pi) &= \sum_{e \in E'} C'_{i,j} = \sum_{(i,j) \in E'} C_{i,j} + \sum_{i=1}^N d_i \pi_i \\ &= \sum_{e \in E'} C'_{i,j} = \sum_{(i,j) \in E'} (C_{i,j} + \pi_i + \pi_j) \end{aligned}$$

حيث  $E'$  هي مجموعه جزئيه من مجموعه الوصلات الكليه  $E$ .

الهدفان الاساسيان من طريقه Held-Karp والتي تستخدم عمليه Lagrangian Relaxation هما:

1. الهدف الرياضي (ايجاد اقصى حد ادنى): قيمه الحل الصحيح لاي مجموعه بيانات في مشكله البائع المتجول تنحصر بين قيمتين, الاولى هي الحد الاعلى وهو قيمه يتم حسابها بشكل سريع (باستخدام خوارزميه جشعه greedy مثلاً) وتكون قيمتها اكبر من الحل الصحيح, اي تحصرها من الاعلى. اما القيمه الثانيه فهيه الحد الادنى, والذي يحصر الحل الحقيقي من الاسفل. والهدف من عمليه Lagrangian Relaxation هو محاوله رفع هذه القيمه لكي تقترب قدر الامكان من الحل الصحيح, حيث يمثل الحد الاعلى السقف الذي لا يمكننا تخطيه رياضياً.

2. الهدف الهندسي (ايجاد قيم التقارب  $\alpha$ -nearness): في مشاكل البائع المتجول الكبيره, حيث يصل عدد البيانات المراد تقليل داله التكلفه بينها الى اعداد كبيره (مثل 10,000 مدينه او اكثر), يصبح البحث المحلي عديم كفاءه, لذلك يجب تقليل عدد الوصلات التي

لا تفيد في إيجاد الحل عن طريق إيجاد اقرب المرشحين لكل مدينه والذين يمثلون الجيران الاقرب والاكثر قابليه لان يكونو في الحل الامثل.

تبدا عمليه التحسين لايجاد اقصى قيمه للحد الادنى في حلقه تتكون من خمس خطوات متكرره:

### 2.3.2 حساب شجره 1-Tree

يعد إيجاد مسار صحيح لمسألة البائع المتجول TSP أمرًا بالغ الصعوبة بسبب قيد صارم: يجب أن يكون حلقه واحدة متصلة حيث تتصل كل مدينة بمدينتين أخريين فقط. ويُعتبر إيجاد بنية ذات وزن أدنى وبهذه الخاصية تحديًا كابوسًا من فئة NP-Hard. تتجاوز شجرة 1-Tree هذه الصعوبة بتخفيف القواعد بما يكفي لتسهيل العمليات الحسابية، مما ينشئ بنية تعد بديلًا مثاليًا لمسار TSP.

ما هو السبب وراء اختيار البنية 1-Tree؟ رياضيًا، شجرة 1-Tree هي بنية بيانية مبنية على  $N$  مدينة وتحتوي على  $N$  وصله بالضبط وحلقة مغلقة واحدة فقط، ويتم ذلك باستخدام حيلة تفكيك بسيطة من خطوتين:

1. اختر مدينة واحدة لتكون "المدينه المميزه" (غالباً المدينه 0). أزلها تمامًا من الخريطة، ثم أنشئ شجرة امتداد دنيا قياسية (MST) عبر المدن المتبقية  $N-1$  باستخدام خوارزمية Kruskal. حيث ان الشجر الممتدة الدنيا تربط جميع النقاط مع بعضها باقل تكلفه ممكنه وبدون انشاء اي حلقه مغلقة.

2. بمجرد العثور على الشجرة الممتدة الدنيا، نعود إلى المدينه 0 ونقوم بتوصيلها قسرًا بالشجرة الممتدة الدنيا الناتجة باستخدام أرخص حافتين متاحيتين لها. بناءً على تكاليفك الحالية. ولأن الشجرة الممتدة الدنيا القياسية لا تحتوي على اي حلقات، فإضافه عقدة إضافية واحدة (المدينه 0) ستضمن إنشاء دورة واحدة بالضبط.

لماذا تعد 1-Tree مرآه مثاليه لمشكله TSP؟ لنلق نظرة على المتطلبات الهيكلية لمسار صالح في مسألة البائع المتجول TSP مقارنة بـ 1-Tree:

- تحتوي جولة TSP الصحيحة على  $N$  من الوصلات، متصلة بالكامل، ولها دورة واحدة فقط، ولكل عقدة درجة تساوي 2 بالضبط.
- تحتوي الشجرة أحادية البعد على  $N$  حافة متصلة بالكامل، ولها دورة واحدة فقط، لكن درجات عقدها يمكن أن تكون أي قيمة (درجة 1 أو أكثر)، وهو عدد الوصلات المتصلة بها.

وهذا يعني أن مسار TSP الصالح هو ببساطة نوع خاص من بنية Tree-1، حيث تكون درجة كل مدينه مساوية لـ 2، كما أن قواعده وبنيته البسيطة تجعل عملية بنائه سهلة للغاية.

لاستخراج الشجرة الممتدة الدنيا Minimum Spaning Tree (MST)، نستخدم خوارزمية Kruskal. بما أننا نبني شجرة أحادية، فإننا نتجاهل مدينه واحده، مما يترك لنا عدد  $N - 1$  مدينه، وفي نظرية الرسوم البيانية (Graph Theory)، تتطلب الشجرة الممتدة الدنيا MST التي تربط مجموعة من النقاط  $N$  دائماً  $N - 1$  حافة بالضبط لربطها جميعاً دون تكوين حلقة مغلقة، وبالتالي سيكون إجمالي عدد الوصلات  $(N - 1) - 1$  (حيث 1- المدينه المميزه التي عُزلت).

كيف تعمل خوارزمية Kruskal؟ أثناء قيام الخوارزمية بالمرور عبر الوصلات المرتبة (من الأرخص إلى الأعلى)، تطرح سؤالاً بسيطاً: "إذا أضفت هذه الوصله، فهل ستُنشئ حلقة مغلقة غير قانونية؟"، يجب هيكّل المجموعة المنفصلة (Disjoint Set) على هذا السؤال فوراً من خلال تتبع المدن المتصلة بالفعل بفروع هيكليه. ولذلك يستخدم مصفوفتين بسيطتين:

- **parents**: يتتبع الجذر المباشر (الأصل) لكل مدينه في مجموعتها، إذا كانت المدينه هي أصلها، فهي جذر مجموعتها المستقلة.

- **ranks**: يتتبع عمق فروع الشجرة لمجموعة ما لضمان أنه عند دمج مجموعتين، يتم توصيل المجموعة الأقصر أسفل المجموعة الأطول.

عند بدء تشغيل الخوارزمية، تكون كل مدينه كياناً منفرداً ومعزولاً تماماً؛ إذ لا تعرف أي مدينه منها المدن الأخرى. وتشكل كل مدينه "دائرة اجتماعية" مستقلة خاصة بها، تتألف من عنصر واحد فقط. وتتولى بنية بيانات "المجموعات المنفصلة" (Disjoint Set Union (DSU إدارة هذه الدوائر باستخدام عمليتين أساسيتين:

- **Find**: يسأل مدينه: "من هو القائد المطلق (الأصل الجذري) لدائرتك الاجتماعيه؟"

- **Union**: يجمع بين دائرتين اجتماعيتين منفصلتين تماماً تحت قيادة واحدة موحدة.

أثناء تكرار الخوارزمية على الوصلات المرتبة، تأخذ معرفي مدينتين وتتحقق من جذورهما (كل منهما في دائرتها الاجتماعيه أو مجموعتها الخاصة). إذا لم تكن جذورهما متساوية، فإنها تأخذ الوصله بينهما بأمان، ثم تدمج المجموعة ذات الترتيب الأصغر في المجموعة ذات الترتيب الأعلى، بحيث يكون جذر المجموعة ذات الترتيب الأعلى هو جذر المجموعة ذات الترتيب الأصغر. أما إذا كانت الجذور متساوية، فإنها تتخطى كليهما لأن لهما مساراً مشتركاً أقصر، وبالتالي لا يحتاجان إلى وصله مباشرة بينهما من شأنها أن تسبب حلقة مغلقة.

### 3.3.2 حساب $G_i$ Subgradient

يمثل التدرج الفرعي الاتجاه الرياضي لأكبر خطأ. لكل مدينة  $i$ , نحسب كم تنحرف درجتها الحالية  $d_i$  في شجرة 1-Tree عن الهدف المراد، حيث في أي رحله TSP فعليه كل مدينة درجتها تساوي 2 تماماً. يمكننا تعريف الخطأ الهيكلي الفردي لأي عقدة  $i$  على النحو التالي:

$$G_i = d_i - 2$$

لفهم كيف يُمثل التدرج الفرعي الاتجاه الرياضي لأكبر خطأ، من المفيد الابتعاد عن البرمجة والنظر إلى هندسة التحسين. إذا كنت تتسلق جبلاً في ضباب كثيف وتريد الوصول إلى القمة بأسرع وقت ممكن، فإنك تنظر إلى الأرض تحت قدميك وتخطو في الاتجاه الذي يصعد فيه المنحدر بأكبر انحدار ممكن. في علم التفاضل والتكامل، يُسمى اتجاه الصعود الأكبر بالتدرج (Gradient - grad). ومع ذلك، فإن رسم دالة الهدف في تحسين Held-Karp ليس منحناً أملساً ومستديراً تماماً. لأن الشجرة أحادية البعد تتكون من وصلات حادة منفصلة، يبدو رسم الدالة وكأنه ماسة متعددة الأوجه أو مثل الهرم متعدد الرؤس. يحتوي على وصلات وزوايا حادة حيث يفشل حساب التفاضل والتكامل القياسي لأنه لا يمكنك حساب مشتقة تقليدية عند نقطة حادة. هنا يأتي دور التدرج الفرعي (Subgradient). حيث أنه تعميم للتدرج للدوال غير الملساء والمتعرجة.

في المرحلة الثانية، يتمثل هدفنا الأول هو جوله صالحه (Valid TSP Tour). في الجولة الصالحة، يكون لكل مدينة هدف بان تكون درجتها تساوي 2 بالضبط. إذا أنتجت خوارزمية Kruskal شجرة 1-Tree وكانت درجة إحدى المدن 4، فإننا نواجه خطأ هيكلياً، أي ان المدينة مكتظة. إذا كانت درجة إحدى المدن تساوي 1، فإننا نواجه خطأ أيضاً، أي ان المدينة معزولة.

إذا كانت المدينة مزدحمة ( $d_i = 5$ )، يكون تدرجها موجباً ( $G_i = 5 - 2 = +3$ ). إذا كانت المدينة منعزلة أو طريق مغلق ( $d_i = 1$ )، يكون تدرجها سالباً ( $G_i = 1 - 2 = -1$ ). وإذا كانت المدينة في حاله دوره TSP مثاليه ( $d_i = 2$ )، ينهار تدرجها نحو الصفر ( $2 - 2 = 0$ ).

لتكن صيغة اجمالي "التكلفة العقابيه" للشجرة الخاصة بنا بالنسبه الى درجه كل مدينة  $W(\pi)$ :

$$\begin{aligned}
W(\pi) &= \sum_{(i,j) \in E'} (C_{ij} + \pi_i + \pi_j) \\
&= \sum_{(i,j) \in E'} C_{ij} + \sum_{(i,j) \in E'} (\pi_i + \pi_j) \\
&= \sum_{(i,j) \in E'} C_{i,j} + \sum_{i \in N} d_i \pi_i
\end{aligned}$$

لننظر كيف يؤثر تغيير عقوبة مدينة واحدة  $\pi_i$  على إجمالي التكلفة المعاقبة  $W(\pi)$ . إذا اعتبرنا وصلات شجرة 1-Tree المختارة ثابتة مؤقتاً، فيمكننا حساب المشتقة الجزئية للتكلفة بالنسبة لعقوبة تلك المدينة:

$$\frac{\partial W}{\partial \pi_i} = d_i$$

يُشير هذا إلى أن معدل التغير الحالي (الميل) لدالة التكلفة على طول المحور  $\pi_i$  يساوي تماماً الدرجة الحالية للعقدة،  $d_i$ . ومع ذلك، فإننا نسعى لتعظيم الحد الأدنى، ولكن يجب علينا القيام بذلك دون السماح "لقيم العقاب" (penalties) بتشويه المخطط البياني إلى ما لا نهاية. وتقتضي مسألة التحسين المزدوج (dual optimization problem) تحقيق توازن في النظام، حيث تقوم خطوة التعديل المستهدفة بضبط الميل بالنسبة للقيد المفروض (أي الدرجة المستهدفة هي 2)؛ وعند طرح القيد المستهدف، يصبح متجه الميول الاتجاهية كما يلي:

$$\nabla W(\pi) \approx \begin{bmatrix} d_1 - 2 \\ d_2 - 2 \\ \vdots \\ d_N - 2 \end{bmatrix} = \begin{bmatrix} G_1 \\ G_2 \\ \vdots \\ G_N \end{bmatrix} = \vec{G}$$

هذا المتجه  $\vec{G}$  هو التدرج الفرعي الخاص بنا.



يخبرنا Subgradient بالاتجاه الذي يجب سلكه لتعديل أخطاءنا البنيوية وماهي النقاط التي علينا تعديلها. على سبيل المثال، إذا كانت المدينة ذات درجة تساوي 5 ( $G_i = +3$ ). يكون الميل ايجابي واتجاه subgradient vector يُوْشر بشده نحو الاتجاه الموجب. ومع التقدم في الحلقات، يسبب ذلك ازدياد قيمه  $\pi_i$  لتلك المدينة بشكل اقوى. وهذا يجعل الوصلات المتصله بتلك المدينة اكبر بالوزن واغلى مع كل حلقه، مما يجبر خوارزمية Kruskal على اقتطاع الوصلات عاليه التكلفه منها.

وعلى العكس تماماً، إذا كانت المدينة ذات درجة منخفضه مثل 1 ( $G_i = -1$ ). في هذه الحاله يُوْشر متجه subgradient نحو الاتجاه السالب. ومع الوقت، ستنخفض قيمه  $\pi_i$  لتلك المدينة. مما يجعل الوصلات المتصله معها اخص في التكلفه، ويجبر ذلك خوارزمية Kruskal على اضافته المزيد من الوصلات لتلك المدينة.

#### 4.3.2 حساب حجم الخطوه $\lambda$ Step Size

عند الرغبه في تصحيح الاخطاء البنيويه عن طريق تعديل قيم العقوبه  $\pi$  لايمكننا ببساطه تغيير قيمها بشكل اعمى؛ فعل ذلك سيؤدي لانحراف النظام بشكل كبير وعشوائي عن القيمه الفضلى. يتم حساب حجم التغير عن طريق استخدام متغير step-size scalar t، مقدار بواسطه Polyak step-size formula :

$$\lambda = \epsilon \frac{UB - L(\pi)}{\sum_{i=1}^N (G_i)^2}$$

حيث:

- $\lambda$  Lambda: المعامل الناتج الذي يحدد شده تعديل قيم عقوبه في الحلقه الحاليه.
- $\epsilon$  Epsilon: هو معامل التخامد التكيفي والذي يبدأ حول عدد حقيقي (2) ويتلاشى نحو الصفر، ويعمل كمضاعف لسرعه او ثقه الخوارزميه.
- $L(\pi)$  Lower Bound: وهو الوزن الحقيقي غير العقابي لشجره 1-Tree الحاليه، ويحسب بالعلاقه التاليه:

$$L(\pi) = W(\pi) - 2 \sum_{i \in N} \pi_i$$

- Upper Bound (UB): هو تقدير للقيمة الحقيقية للسقف, غالبًا ما يحسب باستخدام خوارزميه بحث سريعه مثل nearest neighbor search, حيث ان الفرق بين الحد الاعلى والحد الادنى يسمى ال Optimality Gap.

- المقام  $\sum(G_i)^2$ : مجموع الاخطاء للتربيع, ذلك كان الشكل العام لتوزيع المدن عشوائياً والدرجات بعيدة عن 2, تكبر قيمه المقام, مما يجعل حجم الخطوه اصغر واكثر استقراراً.

### 5.3.2 تحديث قيم العقاب $\pi$

اخيراً, يتم تعديل مصفوفه قيم العقوبات (او يتم تعديلها لكل مدينه) من اجل دوره التاليه, اذا كانت درجه المدينه عاليه, قيمه  $\pi$  خاصتها تزداد. في دوره التاليه, كل الوصلات المتصله معها تصبح "اغلى", يدفع ذلك خوارزميه Kurshal على تجنبها. وبالعكس, المدن المنفيه او ذات الوصله الواحده يحصلون على قيمه تعديل ( $\pi$ ) سالبه, مما يجعل وصلاتهم "ارخص" واكثر جذباً للخوارزميه في دوره التاليه. ويتم ذلك بالمعادله التاليه:

$$\pi_{i_{new}} = \pi_{i_{old}} + \lambda \times G_i$$

## 4.2 الخطوه الثالثه: The Alpha ( $\alpha$ ) Transformation and Candidate Set Generation

في هذه المرحله يحدث السحر. حيث تستخدم شجرة 1-Tree المُحسَّنة كمرشح رياضي لحذف ما يقرب من 95% إلى 99% من الوصلات في الرسم البياني بشكل دائم, مما يحول المشكله من مشكله مستحيله حسابياً إلى سلسله من عمليات البحث المحليه فائقة السرعة.

مفهوم تقارب ألفا (alpha-nearness) في تحليل حساسية الرسم البياني (Graph Sensitivity Analysis) في التحسين التوافقي (Combinatorial Optimization), يتطلب تحديد الوصلات ذات الاحتمالية الأعلى للانتماء إلى مسار البائع المتجول الأمثل يتجاوز المسافة المكانية المجردة. فبينما توفر الأوزان المُعاقبة  $C'_{ij}$  الناتجة عن عمله Held-Karp Relaxation حدًا أدنى ممتازًا (Best Lower Bound), إلا أنها لا تقيس بشكل صريح القيمة الفردية للوصلات غير الشجرية. ولحل هذه المشكله, تعتمد خوارزميه Lin-Kernighan (LKH) على مقياس "حساسية" يُعرف باسم قيم تقارب ألفا (alpha-nearness)

(values).

#### 1.4.2 ما هي قيم ألفا ( $\alpha$ )؟

في نظرية Graph Thoery التقليدية، إذا أردنا معرفة ما إذا كانت إحدى الوصلات جيدة، ننظر إلى مقدار المسافة خاصتها. أما في خوارزمية LKH-3، ننظر إلى قيمة ألفا (المعروفة أيضًا بالتكلفة الزائدة أو قيمة الندم).

تُعرّف قيمة ألفا للوصلة  $(i, j)$  رياضيًا كما يلي: ما مقدار الزيادة في التكلفة الإجمالية لشجرة الحد الأدنى 1-Tree إذا أُجبرنا الخوارزمية على إضافته هذه الحافة تحديدًا؟

#### 2.4.2 التعريف الرياضي لقيمه التقارب (Nearness Value)

لنفترض أن  $L(\pi)$  يمثل إجمالي التكلفة غير العقابيه او الحد الأدنى الحقيقي لشجرة 1-Tree الذي تم ايجاده في نهاية مرحلة تحسين التدرج الفرعي (Subgradient Optimaization). لنفترض أننا نختار وصله عشوائية  $(i, j)$  من المجموعة العامة لجميع الوصلات الممكنة  $E$ ، ونفرض قيدًا صارمًا: يجب على الخوارزمية توليد شجرة أحادية 1-Tree جديدة تتضمن الوصلة  $(i, j)$ . ولنرمز إلى تكلفة هذه الشجرة الأحادية المقيدة (Constrained 1-Tree) حديثًا بـ  $L_{ij}(\pi)$ . تُعرّف قيمة تقارب ألفا للوصلة  $(i, j)$  على أنها الفرق بين هاتين الحالتين:

$$\alpha_{(i,j)} = L_{i,j}(\pi) - L(\pi)$$

بما أن  $L(\pi)$  تمثل الحد الأدنى العام غير المقيد، فإن قيمه الحد الأدنى للشجرة المقيدة  $L_{i,j}(\pi)$  يجب أن تكون أكبر من أو تساوي  $L(\pi)$  تمامًا. لذلك، فإن  $\alpha_{(i,j)}$  دائمًا قيمة غير سالبة ( $\alpha \geq 0$ ).

#### 3.4.2 التفسير الطوبولوجي لقيم ألفا: Topological Interpretation

يوفر مقدار قيمة ألفا للوصلة تقييمًا نظريًا دقيقًا لمدى فائدتها:

•  $\alpha_{(i,j)} = 0$ : تُعدّ الحافة  $(i, j)$  عنصرًا طبيعيًا موجودا في شجرة 1-Tree المثلى. ولا يُغيّر إدراجها (الوصلة) قسراً شيئاً، ما يؤدي إلى انعدام التكلفة الزائدة. وتمثل هذه الوصلات العمود الفقري للشبكة المثلى.

- قيمه  $\alpha(i, j)$  صغيره: لا يقع هذا الخط في شجرة 1-Tree الحالية، ولكن إدخالها بالقوه في البنية لا يحدث سوى زيادة طفيفة في الحد الأدنى العام. تتميز هذه الخطوط بتنافسية عالية من الناحية الرياضية، ولها احتمالية كبيرة للظهور في المسار الأمثل الحقيقي.
  - قيمه  $\alpha(i, j)$  كبيره: يؤدي فرض وجود حافة إلى تعطيل بنية الشجرة 1-Tree بشكل جذري، مما يتسبب في ارتفاع التكلفة الإجمالية بشكل كبير. حتى لو كانت الوصلة قصيرة جغرافياً، فإن قيمة ألفا العالية تشير إلى أنها طريق مسدود طوبولوجياً أو فخ هيكلي.
- باستخدام هذا المقياس، ينتقل LKH من قياس مطلق للمسافة إلى قياس نسبي للضرورة الهيكلية.

#### 4.4.2 الآلية الخوارزمية لحساب قيم ألفا: The Cycle-Exchange Theorem

على الرغم من أن التعريف النظري لتقارب ألفا (أي إدخال حافة في الشجرة 1-Tree وقياس فرق التكلفة الإجمالي) سليم من الناحية المفاهيمية، إلا أن حسابه بطريقة مباشرة غير عملي من الناحية الحسابية. حيث يتطلب استخدام أسلوب البحث الشامل تشغيل خوارزمية Kruskal  $O(n^2)$  مرة، مرة واحدة لكل وصلة نريد حساب قيمه التقارب لها، مما ينتج عنه تعقيد زمني إجمالي هائل.

لحل هذه المشكلة، تعتمد خوارزمية Lin-Kernighan-Helsgaun (LKH) على خاصية أساسية في طوبولوجيا الرسم البياني تُعرف بالدورة الأساسية The Fundamental Cycle. تسمح هذه الخاصية للخوارزمية بحساب قيمة ألفا لأي وصلة في زمن ثابت  $O(1)$ ، بشرط تنفيذ خطوة حسابية مسبقة واحدة عالية الكفاءة سيتم شرحها لاحقاً.

تحتوي الشجرة الممتدة الدنيا MST المكونة من  $N$  عقدة على  $N - 1$  وصلة بالضبط، ولا تحتوي على حلقات مغلقة. وبحسب تعريف طوبولوجيا للشجرة، يوجد مسار واحد متصل وفريد يربط أي عقدتين عشوائيتين،  $i$  و  $j$ . ولنرمز لهذا المسار الفريد بالرمز  $P(i, j)$ . وبحسب التعريف، تسبب إضافة أي وصلة جديدة  $(i, j)$  إلى شجرة موجودة دورة مغلقة واحدة فقط. لاستعادة بنية الشجرة الأصلية ( وإزالة الدورة المغلقة) مع الحفاظ على أقل تكلفة ممكنة، يجب على الخوارزمية إزالة الحافة الأعلى تكلفة الموجودة على مسار الشجرة الأصلي الذي يربط العقدة  $i$  بالعقدة  $j$ .

لإعادة الشجرة إلى البنية الصحيحة، يجب حذف حافة واحدة فقط من هذه الدورة المُشكّلة حديثاً. ولأن هدف الخوارزمية هو إيجاد أقل تكلفة ممكنة لهذه الشجرة المُقيدة حديثاً، يجب عليها استبعاد الحافة الأكثر تكلفة حسابياً ضمن تلك الدورة. وبما أن الوصلة الجديدة  $(i, j)$  مُجبرة على البقاء، يجب اختيار الوصلة المحذوفة من مسار الشجرة الأصلي  $P(i, j)$ . لذا، فإن التغير الصافي في التكلفة الإجمالية للشجرة (وهو تعريف  $\alpha$  تحديداً) هو ببساطة تكلفة

الحافة التي أضفناها، مطروحًا منها تكلفة الحافة التي حذفناها. وهذا يُعطينا صيغة الحساب المباشر:

$$\alpha(i, j) = C'_{ij} - \max_{(u,v) \in P(i,j)} C'_{uv}$$

حيث:

•  $C'_{ij}$  يمثل الوزن العقابي (Penalized Cost  $\pi_i$ ) للوصلة الجديدة التي يتم إدخالها بالقوة في الشجرة.

•  $\max_{(u,v) \in P(i,j)} C'_{uv}$  يمثل وزن الوصلة الأكبر (الأثقل) الموجودة حاليًا على المسار  $P(i, j)$  داخل الشجرة.

#### 5.4.2 استراتيجية الحساب المسبق لأكبر الوصلات في أي مسار فريد

بينما تحصر الصيغة أعلاه الحساب في مسار واحد، فإن فحص كامل الشجرة للعثور على الوصلة الأكبر (الأثقل) لكل زوج  $(i, j)$  على حدة سيظل مكلفًا حسابيًا.

لذلك بدلًا من البحث عن المسار عند الحاجة، تعتمد خوارزمية LKH على الحساب المسبق لهذه الوصلات القصوى للرسم البياني بأكمله مره واحد باستخدام خوارزمية بحث مُحسَّنة عادةً ما تكون خوارزمية البحث بالعرض أولاً أو Breadth-First Search أو خوارزمية البحث بالعمق أولاً (Depth-First Search).

وَتُنَفَّذُ العملية كما يلي: تختار الخوارزمية العقدة  $i$  لتكون "جذر" الشجرة (Tree Root). تبدأ الخوارزمية عملية بحث للخارج على طول فروع الشجرة. عند زيارة العقدة المجاورة  $v$  عبر الوصلة  $(u, v)$ ، فإنها تتعقب باستمرار أثقل وصله تمت رؤيتها حتى الآن. أقصى وزن للمسار إلى العقدة الجديدة  $v$  هو ببساطة أكبر قيمتين: أعلى وزن مسجل حتى العقدة السابقة  $u$ ، أو وزن الحافة المتصلة مباشرة  $(u, v)$ . تسجل الخوارزمية هذه القيمة القصوى في جدول بحث ثابت بحجم  $N \times N$ . رياضياً، تكون علاقة التكرار أثناء الاجتياز كما يلي:

$$MaxPath(i, v) = \max(MaxPath(i, u), C'_{uv})$$

من خلال تحديد جذر مسار عند كل عقدة من العقد  $N$ ، تُنشئ الخوارزمية مصفوفة كاملة  $N \times N$  من أقصى المسارات في زمن  $O(N^2)$  بالضبط. بمجرد إنشاء هذه المصفوفة في الذاكرة، يُصبح حساب قيمة التقارب  $\alpha$  لأي حافة نظرية  $(i, j)$  إلى عملية طرح حسابية بسيطة وفورية.

#### 6.4.2 تبسيط الرسم البياني وبناء مجموعات المرشحين

بينما توفر مصفوفة ألفا درجة حساسية دقيقة رياضياً لكل اتصال محتمل في الشبكة، فإن استخدام هذه المصفوفة  $N \times N$  مباشرةً خلال مرحلة التوجيه يبقى غير عملي حسابياً. تهدف المرحلة الثالثة إلى تحويل قيم تقارب ألفا العددية المستمرة إلى بنية طوبولوجية منفصلة ومتفرقة للغاية تُعرف باسم الرسم البياني للمرشحين Candidate Graph.

تعزل هذه العملية بشكل منهجي الوصلات الأكثر أهمية مع استبعاد الاتصالات غير ذات الصلة إحصائياً بشكل دائم، مما يحد من نطاق البحث القادم من  $O(N^2)$  إلى  $O(N)$ .

#### 7.4.2 التعريف الرياضي لمجموعة المرشحين

لأي عقدة معينة  $i$  في رسم بياني كامل  $G = (V, E)$ ، يشمل الجوار الكامل لها  $N - 1$  وصله ممكنة. تُعرّف مجموعة المرشحين  $C(i)$  على أنها مجموعة فرعية محدودة تمامًا من هذه العقد المتجاورة، تحتوي فقط على  $k$  من أكثر الوصلات الواعدة وفقًا لمصفوفة ألفا.

لإنشاء مجموعته  $C(i)$ ، تُقيّم الخوارزمية مجموعة جميع العقد المتجاورة  $j \in V \setminus \{i\}$ ، وترتبها ترتيباً تنازلياً بناءً على معايير تقييم متعددة المستويات:

1. المقياس الأساسي (تقارب ألفا): يتم تصنيف العقد بشكل أساسي بناءً على قيم ألفا  $\alpha(i, j)$ . يتم وضع الوصلات حيث  $\alpha(i, j) = 0$  (والتي تنتمي أصلاً إلى شجرة 1-Tree المثلى) في أعلى القائمة.

2. كسر التعادل من الدرجة الثانية (الوزن العقابي): إذا كانت وصلات متعددة تشترك في قيمة ألفا أي لها نفس القيمة (وهو أمر شائع للغاية بالنسبة لـ ألفا = 0)، فسيتم فرزها فرعياً حسب وزنها العقابي وفقاً لـ Held-Karp،  $C'_{ij}$ .

3. كسر التعادل من الدرجة الثالثة (المسافة الهندسية): في حالة نادرة من التعادل في الوزن العقابي، تعمل المسافة المكانية الخام  $C_{ij}$  ككسر للتعادل النهائي.

تقوم الخوارزمية بفلتره هذه القائمة المرتبة، مع الاحتفاظ بعدد وصلات أقصى (غالبًا اقرب 5 مرشحين) في أعلى القائمة وتجاهل الباقي.

## 8.4.2 اختيار حد عدد المرشحين: The Theoretical Bound

يمثل حجم مجموعة المرشحين،  $|C(i)|$ ، مفاضلة حاسمة بين الحفاظ على الحل الأمثل وسرعة التنفيذ. إذا كان حجم  $|C(i)|$  كبيرًا جدًا، فإن عامل التفرع لمحرك البحث المحلي اللاحق سيؤدي إلى تضخم هائل في عدد الاحتمالات. أما إذا كان حجم  $|C(i)|$  صغيرًا جدًا، فإن الخوارزمية تُخاطر بحذف وصله تنتمي فعليًا إلى مسار البائع المتجول الأمثل، مما يحرم الحل رياضياً من الوصول إلى الحد الأدنى غير المحلي (Global Minima).

من خلال تحليل تجريبي مُعمّق على مجموعات البيانات في مكتبه TSPLIB، وجد Helsgaun أن مقياس  $\pi$ -nearness Held-Karp دقيق للغاية لدرجة أن الوصلات المثلى الحقيقية تُصنّف دائماً تقريباً ضمن أفضل 5 مراكز. ونتيجةً لذلك، يُفرض المعامل القياسي في LKH-3 حدًا صارمًا قدره:

$$|C(i)| \leq 5$$

من خلال تقييد كل عقدة بعدد مرشحين اقصى يبلغ 5 مرشحين داخل الرسم البياني، يتقلص فضاء البحث إلى حد أعلى يبلغ  $5N$  وصله، مما يجعل التعقيد الزمني لعمليات البحث المحلية خطيًا تمامًا بالنسبة لحجم الرسم البياني.

## 5.2 الخطوه الرابعه: محرك البحث المحلي المتغير k-opt

### 1.5.2 الهدف من البحث المحلي (Local Search)

بينما كان الهدف الاساسي من المرحله الثانيه هو ايجاد اقصى حد ادنى (ارضيه) ممكن، فان هدف مرحله البحث المحلي هو تقليل الحد الاعلى قدر الامكان بحيث يقترب او يلامس الحد الادنى. وبمعنى اخر يمكن توصيف العلاقة الهيكلية بين مراحل تحديد الحدود السابقة وعملية تنفيذ البحث المحلي، من الناحية الرياضية، بأنها عملية تصغير للفجوة بين الحد الادنى (الارضيه) والحد الاعلى (السقف).

تعتمد طرق البحث المحلية التقليدية لحل مسألة البائع المتجول TSP، مثل  $opt-2$  و  $opt-3$ ، على نطاقات طوبولوجية محددة مسبقًا. يُقيّم البحث المحدود مسبقًا جميع التوليفات الممكنة لتبديل  $k$  من الوصلات. ومع ازدياد قيمة  $k$ ، يتزايد التعقيد الحسابي أُسّيًا وفقًا لـ  $O(N^k)$ ، مما يجعل حساب الأعماق العشوائية مستحيلًا في الشبكات الكثيفة. ونتيجةً لذلك، غالبًا ما تتقارب الطرق الثابتة قبل الأوان، فتقع في الحد الأدنى المحلي (Local Minima) حيث لا يُحقق أي تبديل  $k-opt$  صالح تخفيضًا هندسيًا إيجابيًا في اجمالي المسافه، حتى لو تمكن

تبديل من رتبة أعلى (مثل opt-8 أو opt-12) من تجاوز هذا المأزق الطوبولوجي.

يحل أسلوب Lin-Kernighan (LK) هذه المشكلة الهيكلية جذرياً باستبدال تقييم النطاق الثابت بتبديل تسلسلي ذي عمق متغير. بدلاً من تحديد تعقيد عملية التبديل مسبقاً، تقوم الخوارزمية بإنشاء تحركات opt-k المعقدة بشكل ديناميكي، مما يسمح لعمق البحث بالتكيف في الوقت الفعلي بناءً على الوعد الرياضي للبحث.

مع تحديد مساحة البحث رياضياً بواسطة قوائم المرشحين غير الموجهة، تنتقل خوارزميه Lin-Kernighan-Helsgaun (LKH) إلى التوجيه الفعلي. تبدأ الخوارزمية بإنشاء مسار أولي صالح، قد لا يكون مثاليًا (عادةً عبر بحث جشع مثل Depth-First Search عبر وصلات المرشحين). ثم يتحول الهدف إلى تحسين هذا المسار بشكل متكرر باستخدام طريقة البحث المحلي متغير العمق Lin-Kernighan variable-depth local search heuristic.

## 2.5.2 كيف يعمل البحث المحلي مع تبديلات opt-2, top-3, ..., opt-k الثابتة

تعمل خوارزمية البحث المحلي (Local Search) وفق مبدأ التحسين التكراري الاستدلالي (heuristic) ضمن فضاء توافقي متقطع. ولنفترض أن  $T$  تمثل دورة هاميلتونية (Hamiltonian Tour) ممكنة (أي مساراً صالحاً لحل مسألة البائع المتجول - TSP) في المخطط البياني  $G = (V, E)$ ، وأن  $f(T)$  هي دالة التكلفة القياسية التي تحسب إجمالي المسافة الهندسية لهذا المسار:

$$f(T) = \sum_{(u,v) \in T} C_{uv}$$

تُعرف خوارزمية البحث المحلي خريطةً هيكليةً تُسمى الجوار، ويُرمز لها بـ  $N(T)$ . يُمثل الجوار  $N(T)$  المجموعة الكاملة لجميع الدورات الهاميلتونية (Hamiltonian Cycles) الصالحة التي يُمكن اشتقاقها بتطبيق تعديل هيكلية مُحدد (يُشار إليه بالنقل أو التبادل - على المسار الحالي  $T$ ).

يقوم مُحرك التحسين بتقييم المسارات المُرشحة  $N(T)$  بشكلٍ مُنتظم. إذا حقق مسار مُرشح معيار التحسين حيث  $f(T') < f(T)$ ، يُنفذ المُحرك انتقالاً (خطوة)، مُحوّلاً  $T$  إلى  $T'$ . تستمر هذه العملية حتى تصل إلى حالةٍ حيث:

$$\forall T' \in N(T) \quad ; \quad f(T') \geq f(T)$$



عند عتبة الإحداثيات هذه، يكون النظام قد استقر عند قيمة مثلى محلية (Local Optimum) بالنسبة لدالة الجوار  $N$ .

تتضمن عملية التبديل من نوع opt-2 اختيار وصلتين غير متجاورتين من المسار الحالي، يُرمز لهما بالرمزين  $(v_i, v_{i+1})$  و  $(v_j, v_{j+1})$ . ويؤدي حذف هاتين الحافتين إلى تقسيم الدورة إلى مسارين خطيين منفصلين. ولإعادة تشكيل دورة هاملتونية واحدة صالحة، توجد طريقة وحيدة وفريدة لإعادة الربط تتجنب تكوين دورات فرعية منفصلة.

تتطلب عملية إعادة الربط توصيل  $v_i$  بـ  $v_j$  و  $v_{i+1}$  بـ  $v_{j+1}$ . ومن الناحية الطوبولوجية، تؤدي هذه العملية إلى عكس ترتيب التسلسل بالكامل للمسار الوسيط الممتد من الدليل  $i + 1$  إلى الدليل  $j$ .

وتُوسّع خوارزمية opt-3 الاستدلالية حيز البحث التوافقي من خلال معالجة ثلاثة أضلاع مستقلة في آن واحد ضمن المسار النشط.

تتضمن عملية opt-3 اختيار ثلاثة أضلاع متميزة وغير متجاورة من المسار، وهي:  $(v_i, v_{i+1})$  و  $(v_j, v_{j+1})$  و  $(v_k, v_{k+1})$ . وتؤدي إزالة هذه الأضلاع الثلاثة إلى تقسيم المخطط البياني إلى ثلاث سلاسل منفصلة من الرؤوس، يمكننا تسميتها: القطعة A، والقطعة B، والقطعة C.

لإعادة توصيل هذه السلاسل الثلاث المنفصلة في حلقة واحدة صالحة، يوجد عدد  $2^3 \times 1 = 8$  تبديل جبرية إجمالية. يُمثل أحد هذه التباديل المسار الأصلي غير المُعدّل. أما التكوينات السبعة المتبقية الصالحة لإعادة التوصيل، فتُشكل الجوار المحلي الصريح لخطوة واحدة من نوع opt-3، ثم يُعاد توصيل القطع بترتيب تسلسلي جديد.

لأن opt-3 يمكنها تنفيذ عمليات تحويل القطاعات المعقدة التي لا يمكن تقسيمها إلى حركات opt-2 فردية، فإنها تمتلك المرونة الهيكلية اللازمة للهروب من الحد الأدنى المحلي (Local Minima) الذي يشل إجراءات opt-2 الأبسط.

### 3.5.2 قيود خوارزميات k-opt الثابتة

تعتمد طرق البحث المحلية التقليدية، مثل خوارزمية opt-2 أو opt-3، على عمق ثابت ومحدد مسبقاً. إذ لا تُقيّم خوارزمية opt-2 سوى أزواج الوصلات، بينما تُقيّم خوارزمية opt-3 ثلاثيات الوصلات.

يكمن العيب الجوهرى في التحسين بالعمق الثابت Fixed-Depth Optimization في ظاهرة الحد الأدنى المحلي Local Minima. فقد تصل عملية البحث إلى حالة لا يُمكن فيها لأي توزيعه ممكنة من تبديل حافتين أو ثلاث أن تُنتج قيمة  $\Delta$  موجبة. عندئذٍ، تتوقف الخوارزمية،

مُفترضةً أنها وجدت الحل الأمثل. مع ذلك، قد تمتلك عملية تبديل معقدة للغاية، مثل خوارزمية 6-opt أو 8-opt، القدرة على كسر الجمود وتحقيق مسافة أقصر. ولكن تعد برمجة بحث 8-opt ثابتاً أمراً مستحيلاً حسابياً، نظراً لأن عامل التفرع لتبديلات 8 وصلات عبر رسم بياني كبير مرتفع للغاية.

#### 4.5.2 ابتكار Lin-Kernighan: التبديل التسلسلي ذو العمق المتغير

قبل بدء تشغيل محرك البحث المحلي، يفترض تحميل شرطين أساسيين في الذاكرة:

- انشاء الجولة الأساسية: ايجاد دورة هاميلتونية صالحة مُولدة عبر خوارزمية استدلالية سريعة (مثل خوارزمية Nearest Neighbor Search)، حيث يكون لكل عقدة درجة 2 بالضبط.

- مصفوفة المرشحين: من الرسم البياني المنشأ في المرحلة 3. حيث يكون لكل عقدة مصفوفة مُرتبة مسبقاً تحتوي على أفضل 5 أو 6 جيران مميزين لها، مُحددين بناءً على قيم تقارب ألفا.

ومن ثم، تبدأ خوارزمية Lin-Kernighan Local Search (LK) وتتكون من عدة خطوات:

1. الخطوة 1: التهيئة العامة للحلقة الخارجية: يُجري المحرك مسحاً على المستوى الأعلى للعثور على عقدة "ارتكاز" (Anchor Node) أولية يمكنها بدء مسار التحسين.

2. الخطوة 2: الكسر الأول (حذف الأساس): بمجرد اختيار العقدة المرجعية  $t_1$ ، يُنفذ المحرك الخطوة الأولى من مسار تبديلي محتمل لكسر حالة التوازن في المخطط البياني. يتم اختيار عقدة  $t_2$  تكون مجاورة حالياً للعقدة  $t_1$  ضمن المسار النشط. يتم حذف وصله المسار  $x_1 = (t_1, t_2)$  بشكل مؤقت. ثم يسجل الحالة، يدخل المخطط البياني في حالة مؤقتة غير صالحة؛ إذ تنخفض درجة العقدة  $t_1$  إلى 1، وتنخفض درجة العقدة  $t_2$  إلى 1، وتُصنّف العقدة  $t_1$  الآن باعتبارها الطرف المفتوح الدائم للسلسلة.

3. الخطوة 3: التفاعل المتسلسل المتناوب (الحلقة الأساسية): تدخل الخوارزمية الآن في حلقة أعمق، متتبعة مساراً متناوباً من إضافة الوصلات  $(y_i)$  وحذفها  $(x_i)$ . لنفترض أن  $k$  يمثل عمق الخطوة الحالية لتفاعل السلسلة (بدءاً من  $k = 1$ ).

- الخطوة الفرعية A: إضافة وصله: يبحث المحرك عن حافة جديدة مربحة لإضافتها من منظور العقدة المعلقة الحالية  $(t_{2k})$ .

- افتح مصفوفة المرشحين المُرتبة مسبقاً للمرحلة 3 للعقدة  $t_{2k}$ .

- كرر العملية على المرشحين للعثور على عقدة مستهدفة  $t_{2k+1}$  تحقق اختبارات النظر المسبق (انظر للخطوة 4).

- أضف الحافة الجديدة  $y_k = (t_{2k}, t_{2k+1})$  بشكل مبدئي.
- سجل الحالة: تعود العقدة  $t_{2k}$  إلى درجة صالحة 2. تصعد العقدة  $t_{2k+1}$  إلى درجة غير صالحة 3 (ازدحام مروري).
- الخطوة الفرعية B: حذف وصله: يجب على المحرك معالجة حالة التحميل الزائد من الدرجة الثالثة عند العقدة  $t_{2k+1}$  فوراً عن طريق إزالة حافة قديمة.
- حدد حافتي المسار الأصليتين المتصلتين بالعقدة  $t_{2k+1}$ .
- اختر إحدى هاتين الحافتين لحذفها، ويُرْمَز لها بـ  $x_{k+1} = (t_{2k+1}, t_{2k+2})$  وهي الوصلة التي تفك الحلقة المغلقة المستحدثه بعد اضافته الوصلة الاخيره، وفي الكود يتم ذلك بحذف الوصلة التي تؤدي الى العقده التي اتينا منها دون الحاجه الى استخدام خوارزميه بحث بطيئه. (ملاحظة: لا يمكنك حذف الحافة  $y_k$  التي أضفتها للتو في الخطوة السابقة).
- احذف الحافة  $x_{k+1}$  مؤقتاً.
- سجل الحالة: تعود العقدة  $t_{2k+1}$  إلى درجة صالحة 2. تنخفض درجة العقدة  $t_{2k+2}$  المفصلة حديثاً إلى 1.

4. الخطوة 4: The Lookahead Tests: خلال الخطوة الفرعية A، تُرفض العقدة المرشحة  $t_{2k+1}$  فوراً إذا أخفقت في أي من اختبارات النظر السبق الثلاثة التالية:

- معيار الربح: يقوم المحرك بحساب الربح التراكمي الجاري  $G_k$  عبر السلسلة النشطة:

$$G_k = \sum_{i=1}^k (Cost(x_i) - Cost(y_i))$$

يجب أن يكون الشرط  $G_k > 0$  صحيحاً تماماً. إذا تجاوزت تكلفة الوصلات المُضافة وفورات الوصلات المقطوعة، يتم إنهاء الفرع فوراً.

- The Disjoint History Check: لا يمكن أن تكون الحافة  $y_k$  حافة تم حذفها بالفعل في وقت سابق في سلسلة التفاعل المحددة هذه، ولا يمكن أن تكون حافة موجودة حالياً في الجولة الأصلية.

- حدود الحلقة الهيكلية: يجب أن تكون عملية حذف الوصلة  $x_{k+1}$  —المُختارة في الخطوة الفرعية (B)— قادرةً في النهاية على إغلاق مسار صالح؛ فإذا أدى حذفها إلى تحويل المخطط إلى وضع لا يمكن تداركه، يُستبعد هذا الخيار.

5. الخطوة 5: The Hypothetical Closure Check: مباشرةً بعد حذف الوصلة  $x_{k+1}$  في الخطوة الفرعية (B)، يحتوي المخطط (الرسم البياني) على عقدتين فقط بدرجة اتصال تساوي 1: وهما عقدة الارتكاز البادئة  $t_1$  وعقدة النهاية الحالية  $t_{2k+2}$ ؛ بينما تكون جميع العقد الأخرى مستقرة مؤقتاً عند درجة اتصال تساوي 2.

يختبر المحرك وصله إغلاق افتراضية تصل مباشرةً بين عقدة النهاية الحالية ونقطة البداية:  $y_{\text{close}} = (t_{2k+2}, t_1)$ . ثم يتحقق المحرك مما إذا كانت إضافة الحافة  $y_{\text{close}}$  ستشكل دورة هاميلتونية واحدة صالحة تتضمن جميع العقد البالغ عددها  $N$  (مع ضمان عدم نشوء مسارات فرعية معزولة أو منفصلة عن غير قصد).

سجل العمليات المؤقت: إذا كانت الطوبولوجيا صالحة، يحسب المحرك الطول الصافي للمسار. وإذا كان هذا التكوين أقصر من أفضل مسار معروف، تُحفظ سلسلة عمليات التبديل الكاملة الخاصة به في سجل مؤقت باعتبارها "الحركة الأفضل حالياً" (Current Best Move).

قرار التكرار: تتم زيادة قيمة  $k$  (أي  $k \leftarrow k + 1$ )، ثم العودة إلى الخطوة 3 لمحاولة توسيع السلسلة بشكل أعمق بحثاً عن مكاسب أكبر، شريطة استيفاء شروط الخطوة 4 (The Lookahead Tests).

6. الخطوة 6: تنفيذ الحركة وإعادة تعيين الرسم البياني: عندما تصل سلسلة التفاعلات إلى طريق مسدود (عندما يستنفد جميع المرشحين خياراتهم أو ينخفض معيار الربح إلى ما دون الصفر)، ينتهي فرع البحث العميق.

يراجع المحرك سجله المؤقت. إذا لم تُسجّل أي تحسينات صالحة خلال دورة حياة السلسلة بأكملها، تُهمل التعديلات (أي تُسترجع)، وينتقل المحرك إلى متغير العقدة التالي في الحلقة الخارجية.

إذا سُجّل تحسين صالح، يُثبّت المحرك "أفضل حركة حالية" بشكل دائم في مصفوفة الجولة الرئيسية. تُمسح تخطيطات الوصلات القديمة من الذاكرة، وتُسجّل الوصلات الجديدة، ويُعيد المحرك ضبط النظام بالكامل.

7. الخطوة 7: الاستقرار الشامل (الحالة النهائية): تكتمل الخوارزمية بأكملها فقط عندما يحقق النظام استقراراً تاماً أي:

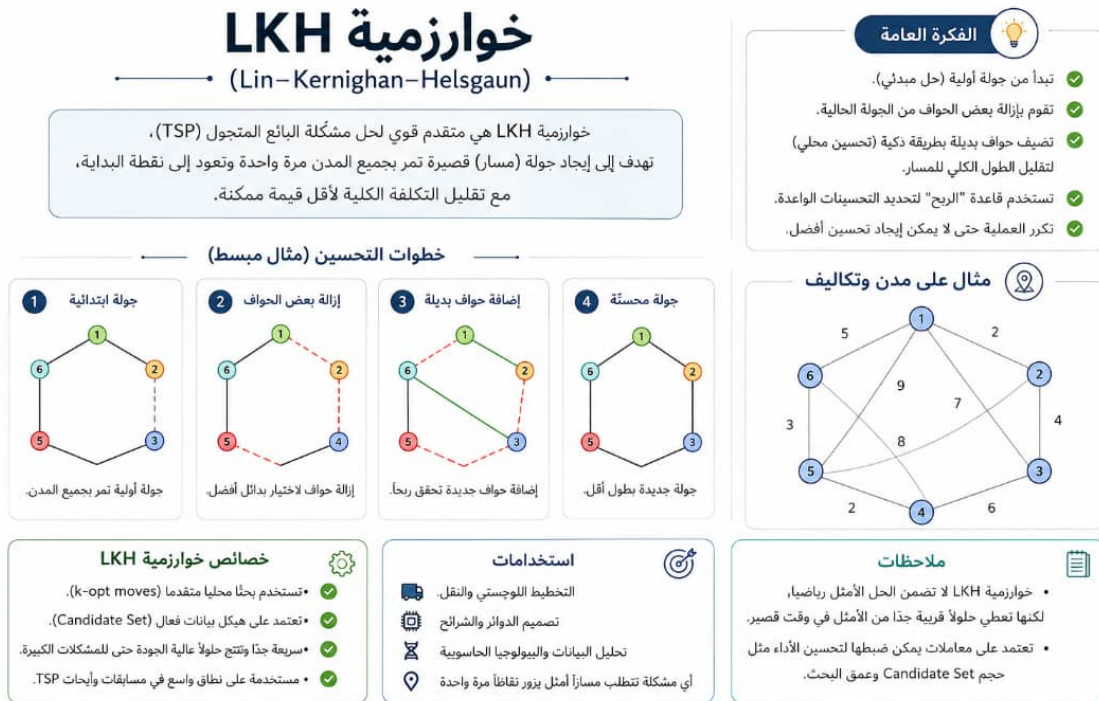
- تنتقل الحلقة الخارجية عبر العقد، بدءاً من العقدة 0 والعقدة 1، ووصولاً إلى العقدة  $N - 1$ .
- تحاول كل عقدة بدء تفاعل متسلسل، وتُسفر جميع عمليات التنفيذ (البالغ عددها  $N$ ) عن فشل في تحقيق أي تحسين.

• عندئذٍ، يخرج النظام من حلقة المعالجة ويُخرج المسار المحسّن للغاية.

## 6.2 Backtracking and the LKH-3 Extension

بينما اقتصر خوارزمية LK الأصلية على التحركات المتسلسلة في مسارات متناوبة بسيطة، يُحسّن إطار عمل LKH الخاص بكيلد هيلسغاون هذه البنية من خلال دمج آليات تراجع Backtracking واسعة النطاق.

إذا لم يُسفر تسلسل ما عن مسار مُحسّن وتجاوز حد الربح الإيجابي، فإن خوارزمية LKH-3 تتراجع بشكل منهجي لأعلى شجرة البحث، مُستبدلةً وصلاتاً مرشحةً بديلة (مثل استبدال  $y_i$  بحافة محلية مرشحة مختلفة) لاستكشاف الفروع المتوازية. علاوة على ذلك، ولأن اختيار  $y_i$  يخضع حصرياً لمجموعات المرشحين ذات التقارب ألفا المتفرقة من المرحلة الثالثة، فإن محرك العمق المتغير هذا قادر على تنفيذ عمليات تبادل متسلسلة تتجاوز عمق  $k = 50$  بسهولة. يمنح هذا التوليف بين ترشيح ألفا والتكرار ذي العمق المتغير الخوارزمية قدرةً لا مثيل لها على تجاوز الحد الأدنى المحلي العميق وعزل الحل الأمثل العالمي الحقيقي.



تم تطويرها بواسطة: Helsgaun (1998) وهي من أقوى خوارزميات حل TSP في العالم.

### 3 Real-World Applications of LKH-3

في حين تُعامل "مسألة البائع المتجول" (Traveling Salesperson Problem) ونُسخها المتعلقة بمركبات متعددة غالباً بوصفها تجريدات رياضية بحتة، فإن خوارزمية الحل LKH-3 التي طورها Keld Helsgaun تُستخدم عالمياً ضمن بعض أكثر منظومات الحوسبة والخدمات اللوجستية تعقيداً على وجه الأرض. ومن خلال تعميم إطار عمل "المسار التبادلي المتسلسل" (sequential alternating path) ليتسنى له التعامل مع أكثر من 40 قيداً توافقياً متميزاً (مثل النوافذ الزمنية، وحدود السعة، وتنسيق حركة المركبات المتعددة)، ينجح برنامج LKH-3 في سد الفجوة بين علوم الحاسوب النظرية وبين تعزيز كفاءة المؤسسات الضخمة التي تُقدّر قيمتها بمليارات الدولارات.

#### 1.3 الخدمات اللوجستية في التجاره الالكترونيه وتحسين النقل (Amazon, FedEx, UPS)

إن التطبيق الأكثر إلحاحاً وأهمية لـ LKH-3 هو في الخدمات اللوجستية لسلاسل التوريد العالمية، وتحديداً معالجة مشكلة توجيه المركبات ذات السعة المحدودة مع النوافذ الزمنية (CVRPTW).

- بنية أمازون التحتية لـ Last-Mile: تعتمد أمازون بشكل كبير على أطر عمل LKH لحساب مصفوفات التوجيه الأمثل والتحقق من صحتها لأسطول التوصيل التابع لها. وفي تحدياتها البحثية العامة لتوجيه الـ Last-Mile، أصبح التكامل الهجين بين التعلم الآلي وحلول LKH المعيار الذهبي لربط المسارات الهيكلية بسلوك السائقين في العالم الحقيقي. علاوة على ذلك، تستخدم خوارزمية LKH-3 بنشاط لحل نماذج التوصيل المعقدة للغاية من الجيل التالي، مثل مسألة البائع المتجول مع طائرات بدون طيار متعددة (TSP-mD)، ومزامنة الشاحنات الأرضية مع عمليات تسليم الطائرات بدون طيار ذاتية القيادة.

- شركات الشحن العالمية (FedEx, UPS): بالنسبة لشركات الشحن التقليدية، فإن توفير 1% فقط من إجمالي المسافة المقطوعة عبر آلاف المسارات يمنع انبعاث ملايين الأطنان من الكربون ويوفر تكاليف وقود هائلة. تستخدم هذه الشركات نظام LKH-3 كمحرك توجيه إنتاجي للمراكز الإقليمية الكبيرة، وكأساس تقييمي دقيق لتدريب خوارزميات الإرسال الخاصة بها القائمة على التعلم العميق في الوقت الفعلي (Real Time).

### 2.3 تصنيع أشباه الموصلات وتحديد مسارات حركة الروبوتات (Intel, TSMC)

يتجاوز دور خوارزمية LKH-3 مجرد نقل المركبات عبر الخرائط الجغرافية، إذ تعالج اختناقات حرجة تتعلق بتحسين المادي على المستوى المجهرى في مجال هندسة العتاد.

- حفر لوحات الدوائر المطبوعة عالية الكثافة: تتطلب لوحات الدوائر المطبوعة الحديثة آلات CNC أو ليزرات عالية الطاقة لحفر عشرات الآلاف من الثقوب الدقيقة لتوصيل المكونات. رسم مسار الليزر أو الذراع الميكانيكية هو مسألة حل مشكلة البائع المتجول الكلاسيكية، وهي مسألة إقليمية بحته.

- زيادة الإنتاجية إلى أقصى حد: في مصانع التصنيع مثل TSMC أو Intel، يُترجم توفير أجزاء من الملي ثانية لكل ثقب عبر ملايين الرقائق إلى مكاسب إنتاجية هائلة. ولأن خوارزمية LKH-3 قادرة على تحقيق فجوات مثالية تقل عن 0.05% على الرسوم البيانية التي تحتوي على ما يصل إلى مليون عقدة، فإنها تُستخدم لإنشاء مسارات تنفيذ ثابتة ومُحسنة للغاية لأنظمة الروبوتات في المصانع.

### 3.3 خدمات حجز المركبات ووسائل التنقل حسب الطلب ديناميكياً (Uber, Grab, Meituan)

في عالم "اقتصاد" الطلب الفوري، تتغير مشاكل التوجيه بشكل ديناميكي كل ثانية. ويتعين على الشركات حل مشكلة الاستلام والتسليم مع مراعاة النطاق الزمني (PDPTW) على نطاق واسع.

- تجميع الرحلات وتجميع الطلبات: عندما تقوم منصات مثل Uber أو Grab بتجميع عدة ركاب في مركبة مشتركة واحدة، يتعين على النظام التقني الأساسي حساب التسلسل الأمثل لنقاط الصعود والنزول بشكل فوري، مع الالتزام بحدود صارمة للحد الأقصى لتأخير الركاب.

- التحقق من صحة الخوارزمية الأساسية: نظراً لأن بيانات الإنتاج تتطلب أوقات استجابة بالمللي ثانية، فإن هذه المنصات تُشغل خوارزميات جشعة مُصممة خصيصاً على خوارزميات النشطة. مع ذلك، يُستخدم LKH-3 في مسار هندسة الأنظمة غير المتصلة بالإنترنت لإجراء محاكاة شاملة على بيانات القياس عن بُعد التاريخية، ما يجعله المعيار المعماري الأمثل للتحقق من أن خوارزم الوقت الفعلي لا تنحرف نحو سلوكيات توجيه دون المستوى الأمثل.

### 4.3 المعلوماتية الحيوية المتقدمة و Genome Assembly (المرافق المركزية لعلم Genome)

يتمثل أحد التطبيقات غير المكانية المثيرة للاهتمام لخوارزمية LKH-3 في مجال البيولوجيا الحاسوبية، وتحديدًا في تجميع الجينوم (Genome) من الصفر (De Novo Genome Assembly).

- مشكلة أقصر سلسلة فائقة مشتركة: عند تسلسل الحمض النووي، لا تستطيع الأجهزة الحديثة قراءة الكروموسوم من طرف إلى طرف؛ بل تقوم بتقسيمه إلى ملايين الأجزاء الصغيرة المتداخلة (القراءات). ويمكن نمذجة إعادة بناء تسلسل الجينوم الأصلي عن طريق زيادة أزواج التداخل إلى أقصى حد رياضياً كمسألة البائع المتجول غير المتناظرة (ATSP).

- حل الرسوم البيانية غير المنظمة: إن قدرة LKH-3 على التعامل بكفاءة مع تكاليف الحواف الموجهة وقيود التسلسل المعقدة تجعلها أداة فعالة للغاية لخوارزميات المحاذاة، مما يساعد فرق المعلوماتية الحيوية على ترتيب تسلسلات النيوكليوتيدات المجزأة في خرائط جينومية عالية الدقة.

## Conclusion: The Engineering Legacy of Lin-Kernighan 4

ختاماً، يُثبت الانتشار العالمي لخوارزمية LKH-3 أن بنية  $opt_k$ -المتغيرة تُعد واحدة من أكثر إنجازات خوارزميات البحث المحلي (local search heuristics) رسوخاً وديمومة. وسواء تعلق الأمر بتحديد مسار شاحنة داخل مدينة مزدحمة، أو توجيه شعاع ليزر عبر رقاقة سيليكون، أو تجميع سلاسل الحمض النووي البشري، فإن الآليات الجوهرية المتمثلة في بناء المسارات التبادلية، والتحقق من درجات العقد، وتجنب القيم الصغرى المحلية (local minima)، تظل تشكل حجر الزاوية في عمليات التحسين الصناعي الحديثة.